

Online Reinforcement Learning in Episodic MDPs

Mohammad Sadegh Talebi

m.shahi@di.ku.dk

Department of Computer Science



Recap: Finite-Horizon MDP

Let $H \in \mathbb{N}$ and define $[H] := \{1, \dots, H\}$.

A finite-horizon MDP with (fixed) horizon H is a tuple

$$M = (\mathcal{S}, \mathcal{A}, \{P_h\}_{h \in [H]}, \{R_h\}_{h \in [H]})$$

- **State-space** $\mathcal{S}$ (with size S)
- **Action-space** $\mathcal{A}$ (with size A)
- **Transition function** P_h : Selecting $a \in \mathcal{A}$ in $s \in \mathcal{S}$ at time step $h \in [H]$ leads to a transition to s' with probability $P_h(s'|s, a)$.
 $P_h(\cdot|s, a)$ is a probability distribution over $\mathcal{S}$, i.e.,

$$\sum_{s'} P_h(s'|s, a) = 1$$

- **Reward function** R_h : Selecting $a \in \mathcal{A}$ in $s \in \mathcal{S}$ at time step $h \in [H]$ yields a reward $r \sim R_h(s, a)$.

Define $P := \{P_h\}_{h \in [H]}$ and $R := \{R_h\}_{h \in [H]}$. Then

$$M = (\mathcal{S}, \mathcal{A}, P, R, H)$$



Recap: Interaction with MDP

An **agent** interacts with the MDP for H rounds.

At each time step $t = 1, 2, \dots, H - 1$:

- The agent observes the current state s_t and takes an action $a_h \in \mathcal{A}$
- MDP decides a reward $r_h := r_t(s_h, a_h)$ and a next state $s_{h+1} \sim P_{\mathbf{h}}(\cdot | s_h, a_h)$
- The agent receives r_h (any time in step h before start of $h + 1$)

At time step $t = H$:

- The agent is in a **terminal state** s_H , receives a reward $r_H(s_H)$, and takes no action.

This interaction produces a trajectory (or history)

$$\mathcal{D} = (s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_{H-1}, a_{H-1}, r_{H-1}, s_H, r_H)$$

The goal is to maximize the total expected sum of rewards

$$\max_{\text{strategies}} \mathbb{E} \left[\sum_{h=1}^{H-1} r_{\mathbf{h}}(s_h, a_h) + r_{\mathbf{H}}(s_H) \right]$$



Recap: Optimal Value

A Markov deterministic policy is a mapping $\pi : \mathcal{S} \times [H] \rightarrow \mathcal{A}$.

The value function of π is a mapping $V^\pi : \mathcal{S} \times [H] \rightarrow \mathbb{R}$ with

$$V^\pi(s, h) := \mathbb{E}^\pi \left[\sum_{t=h}^{H-1} r_t(s_t, a_t) + r_H(s_H) \middle| s_h = s \right].$$

where $\mathbb{E}^\pi$ indicates expectation over trajectories generated by π .

Solving a finite-horizon MDP M amounts to solving, for all $s \in \mathcal{S}$,

$$\sup_{\pi \in \Pi^{\text{HR}}} V^\pi(s, 1) = \max_{\pi \in \Pi^{\text{MD}}} V^*(s, 1) := V^*(s, 1),$$

- V^* is called the optimal value function.
- We are particularly interested in $V^*(s, 1)$ —i.e., the reward accumulated over H steps starting from s .
- Any policy achieving $V^*(s, 1)$ for all s is called **optimal** and denoted by π^* .



Bellman Optimality and Dynamic Programming

Theorem

The optimal value function V^* obeys *optimal Bellman equations*:

$$V^*(s, H) = R_H(s)$$

$$V^*(s, h) = \max_{a \in \mathcal{A}} \left(R_h(s, a) + \sum_{x \in \mathcal{S}} P_h(x|s, a) V^*(x, h+1) \right), \quad \forall h = H-1, \dots, 1$$

V^* and π^* can be computed by **Dynamic Programming (DP)**.

- **initialization:** For all terminal states s : $V^*(s, H) = R_H(s)$
- **for** $h = H-1, \dots, 1$

$$\forall s: \quad V^*(s, h) = \max_{a \in \mathcal{A}} \left(R_h(s, a) + \sum_{x \in \mathcal{S}} P_h(x|s, a) V^*(x, h+1) \right)$$

$$\pi^*(s, h) \in \operatorname{argmax}_{a \in \mathcal{A}} \left(R_h(s, a) + \sum_{x \in \mathcal{S}} P_h(x|s, a) V^*(x, h+1) \right)$$

output: π^*



Bellman Optimality and Dynamic Programming

Theorem

The optimal value function Q^* obeys *optimal Bellman equations*:

$$Q^*(s, a, H) = R_H(s)$$

$$Q^*(s, a, h) = R_h(s, a) + \sum_{x \in \mathcal{S}} P_h(x|s, a) \max_{a'} Q^*(x, a', h+1), \quad \forall h = H-1, \dots, 1$$

Q^* and π^* can be computed by **Dynamic Programming (DP)**.

- **initialization:** For all terminal states s : $Q^*(s, a, H) = R_H(s)$
- **for** $h = H-1, \dots, 1$

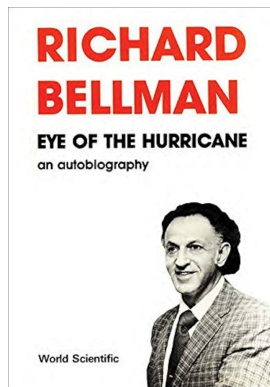
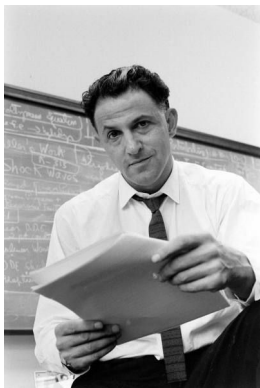
$$\forall s: \quad Q^*(s, a, h) = R_h(s, a) + \sum_{x \in \mathcal{S}} P_h(x|s, a) \max_{a' \in \mathcal{A}} Q^*(x, a', h+1)$$

$$\pi^*(s, h) \in \operatorname{argmax}_{a \in \mathcal{A}} Q^*(s, a, h)$$

output: π^*



Richard Bellman (1920-1984)



Online Episodic RL: Setting and Performance Measure

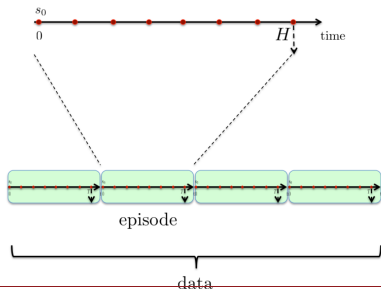


Online Episodic RL

An agent interacts with an episodic MDP $M = (\mathcal{S}, \mathcal{A}, P, R)$ for K episodes (potentially unbounded), and the state is reset at the beginning of each episode.

For each episode $k = 1, \dots, K$:

- s_1 is chosen by Nature
- For timesteps $h = 1, \dots, H$
 - The agent observes $s_{k,h}$ and takes an action $a_{k,h} \in \mathcal{A}$
 - M decides a reward $r_{k,h} \sim R_h(s_{k,h}, a_{k,h})$ and a next-state $s_{k,h+1} \sim P_h(\cdot | s_{k,h}, a_{k,h})$
 - The agent receives $r_{k,h}$



Online Episodic RL

An agent interacts with an episodic MDP $M = (\mathcal{S}, \mathcal{A}, P, R)$ for K episodes (potentially unbounded), and the state is reset at the beginning of each episode.

For each episode $k = 1, \dots, K$:

- s_1 is chosen by Nature
- For timesteps $h = 1, \dots, H$
 - The agent observes $s_{k,h}$ and takes an action $a_{k,h} \in \mathcal{A}$
 - M decides a reward $r_{k,h} \sim R_h(s_{k,h}, a_{k,h})$ and a next-state $s_{k,h+1} \sim P_h(\cdot | s_{k,h}, a_{k,h})$
 - The agent receives $r_{k,h}$

M is unknown (beyond $\mathcal{S}$ and $\mathcal{A}$), and the goal is to maximize $\sum_{k=1}^K \sum_{h=1} r_{k,h}$ (in expectation) using collected experience (history).

The history up to episode k (i.e., data available at the beginning of k):

$$\mathcal{D}_k = \{(s_{j,1}, a_{j,1}, r_{j,1}, \dots, s_{j,H}, a_{j,H}, r_{j,H})\}_{j=1, \dots, k-1}$$



Need for balancing exploration and exploitation.

Regret: Episodic RL

In online episodic RL, the **Regret** of an algorithm $\mathbb{A}$ is the difference between cumulative reward of the optimal policy π^* (oracle) and that gathered by $\mathbb{A}$:

$$\mathfrak{R}(\mathbb{A}, K) := \sum_{k=1}^K V^*(\mathbf{s}_{1,k}, 1) - \sum_{k=1}^K V^{\pi_k}(\mathbf{s}_{1,k}, 1)$$

- π_k is the agent $\mathbb{A}$'s policy executed in episode k :

$$\pi_k = \mathbb{A}(\mathcal{D}_k)$$

where $\mathcal{D}_k$ denotes the history up to beginning of episode k .

- Both oracle and $\mathbb{A}$ are reset to a fixed initial state at the beginning of each episode, i.e.,

$$s_{1,k} = s_{1,k}^*$$

- When each episode is of fixed length H , we have $T = KH$.

Regret is directly connected to the agent's goal, namely maximizing

$$V^{\pi}(s_1, 1) = \mathbb{E} \left[\sum_{h=1}^H r(s_h, a_h) + r_H(s_H) \right]$$

Alternatively, the objective of the agent is to **minimize the regret**.



No-Regret Algorithm

- $\mathfrak{R}(\mathbb{A}, K)$ is a r.v., and we wish to control it **in expectation** or **its deviation with high probability**.
- An algorithm $\mathbb{A}$ is a **no-regret learning algorithm** if either

$$\mathbb{E}[\mathfrak{R}(\mathbb{A}, K)] = o(K) \quad \text{or} \quad \mathfrak{R}(\mathbb{A}, K) = o(K) \quad \text{w.h.p.}$$

No-Regret Algorithm

An algorithm $\mathbb{A}$ is called **no-regret** if there exists a deterministic function f with $\frac{f(T)}{T} \rightarrow_{T \rightarrow \infty} 0$, such that one of the following holds:

$$\mathbb{E}[\mathfrak{R}(\mathbb{A}, K)] \leq f(K)$$

$$\mathfrak{R}(\mathbb{A}, K) \leq f(K) \quad \text{with high probability.}$$

- f could be MDP-dependent but must be deterministic.
- Note that a high-probability bound on $\mathfrak{R}(\mathbb{A}, K)$ implies a bound on $\mathbb{E}[\mathfrak{R}(\mathbb{A}, K)]$, but not the other way around.



Warm-up: A Simple Algorithm



Empirical MDP

For any $t \geq 1$, define

- $N_k(s, a, s', h)$: number of visits, up to episode k , to (s, a) at step of h followed by a visit to s'

$$N_k(s, a, s', h) = \sum_{i=1}^{k-1} \mathbb{I}\{s_{i,h} = s, a_{i,h} = a, s_{i+1,h} = s'\}$$

- $N_k(s, a, h) = \sum_{s' \in \mathcal{S}} N_k(s, a, s', h)$

Empirical Estimator for P :

$$\forall s' \in \mathcal{S}: \quad \hat{P}_{k,h}(s'|s, a) = \begin{cases} \frac{N_k(s, a, s', h)}{N_k(s, a, h)} & \text{if } N_k(s, a, h) > 0 \\ \frac{1}{S} & \text{otherwise} \end{cases}$$

Empirical Estimator for R :

$$\hat{R}_{k,h}(s, a) = \frac{1}{N_k(s, a, h)} \sum_{i=1}^{k-1} r_{i,h} \mathbb{I}\{s_{i,h} = s, a_{i,h} = a\}$$



Warm-up: Certainty Equivalence

Certainty Equivalence (CE) Principle: In an unknown world, act as if your estimates equals their true values.

Synthesis of CE:

- Compute estimates $\hat{P}_k$ and $\hat{R}_k$ using collected data $\implies$ The empirical MDP:

$$\widehat{M}_k = (\mathcal{S}, \mathcal{A}, \hat{P}_k, \hat{R}_k)$$

- Find $\hat{\pi}_k^*$, the optimal policy of $\widehat{M}_k$. Then $a_{k,h} = \hat{\pi}_k^*(s_{k,h}, h)$.

$\implies$ No exploration-exploitation tradeoff. Hence, $\text{Regret}(K) \propto K$.

CE + trembling hand: At episode k , at timestep h ,

$$a_{k,h} = \begin{cases} \hat{\pi}_k^*(s_{k,h}, h) & \text{w.p. } 1 - \varepsilon_k \\ \text{chosen uniformly at random over } \mathcal{A} & \text{w.p. } \varepsilon_k \end{cases}$$

No-regret for suitably chosen ε_k —E.g., $\varepsilon \propto \frac{1}{\sqrt{k}}$.



A Simple Algorithm: CE + Trembling Hand

- **input:** $(\varepsilon_k)_{k \geq 1}$
- **initialization:** $\mathcal{D} = \{\}$
- **for** episodes $k = 1, 2, \dots$ **do**
 1. *Policy Selection*
 - Update counters $N(s, a, h)$ using $\mathcal{D}$
 - Compute estimates $\hat{P}$ and $\hat{R}$ using $\mathcal{D}$
 - $\pi_k = \text{DP}(\hat{M})$ where $\hat{M} = (\mathcal{S}, \mathcal{A}, \hat{P}, \hat{R})$
 2. *Policy Execution*
 - **for** timesteps $h = 1, \dots, H$ **do**
 - Take action $a_h = \pi_k(s_h, h)$
 - Receive reward $r_h \sim R_h(s_h, a_h)$ and next-state $s_{h+1} \sim P_h(\cdot | s_h, a_h)$
 - $\mathcal{D} \leftarrow \mathcal{D} \cup \{(h, s_h, a_h, r_h)\}$



UCRL2: Upper Confidence Reinforcement Learning



The simple algorithm implements

the certainty equivalence principle + exploration.

- Intuitive design (+)
- For suitable ε_t , it becomes no-regret (+)
- Tuning ε_t is not easy, and may require prior knowledge to obtain sublinear regret (−)
- Weak empirical performance (−).

We need more powerful principle.



Optimism Principle

Optimism (in the Face of Uncertainty)

- A well-known principle in balancing **exploration-exploitation** in bandits and online RL dating back to (Lai & Robbins, 1985).

The Optimism Principle (Lai & Robbins, 1985)

In an unknown world, consider a world model (among **plausible** ones) that is most desired (in terms of rewards)!

Intuition:

- If the chosen action is optimal $\implies$ no penalty
- If sub-optimal $\implies$ reducing uncertainty



Optimism Principle

Optimism (in the Face of Uncertainty)

- A well-known principle in balancing **exploration-exploitation** in bandits and online RL dating back to (Lai & Robbins, 1985).

The Optimism Principle (Lai & Robbins, 1985)

In an unknown world, consider a world model (among **plausible** ones) that is most desired (in terms of rewards)!

In bandits: Replace unknown mean rewards by their UCBs (e.g., UCB1 (Auer et al., 2002)).

In MDPs: It is more tricky!

This lecture: A no-regret algorithm (UCRL2) based on Optimism.



UCBVI

UCBVI = Upper Confidence Bound Value Iteration

- UCBVI was introduced in (Azar et al., 2017).
- A **model-based** RL algorithm that relies on optimistic value functions.
- Optimism is implemented by adding suitable exploration bonuses to the Q-value function.



UCBVI: Idea

Consider the empirical MDP: $\widehat{M}_k = (\mathcal{S}, \mathcal{A}, \widehat{P}_k, \widehat{R}_k)$

Idea: To encourage exploration, we perturb $\widehat{M}_k$ by adding an **exploration bonus** b_k to the rewards

$$\widetilde{M}_k = (\mathcal{S}, \mathcal{A}, \widehat{P}_k, \widehat{R}_k + b_k)$$

and then selecting π_k as the optimal policy in $\widetilde{M}_k$.

- b_k is derived by suitable concentration inequalities
- For instance, using Hoeffding's concentration:

$$b_k(s, a, h) = \frac{8H \log(HSAk/\delta)}{\sqrt{N_k(s, a, h)}}$$

- Readily an elegant idea, but may suffer from unnecessarily large optimism

$$Q_k(s, a, h) = \widehat{R}_{k,h}(s, a) + b_k(s, a, h) + \sum_{x \in \mathcal{S}} \widehat{P}_{k,h}(x|s, a) V_k(x, h+1)$$

with $V_k(x, h) := \max_{a \in \mathcal{A}} Q_k(x, a, h)$.



UCBVI: Idea

Perturbed MDP using bonus exploration b : $\widetilde{M}_k = (\mathcal{S}, \mathcal{A}, \widehat{P}_k, \widehat{R}_k + b_k)$

Q-value in $\widetilde{M}_k$:

$$Q_k(s, a, h) = \widehat{R}_{k,h}(s, a) + b_k(s, a, h) + \sum_{x \in \mathcal{S}} \widehat{P}_{k,h}(x|s, a) V_k(x, h+1)$$

with $V_k(x, h) := \max_{a \in \mathcal{A}} Q_k(x, a, h)$.

A Better Proposal. To mitigate the over-optimism we refine Q-values:

$$\begin{cases} Y = \widehat{R}_{k,h}(s, a) + b_k(s, a, h) + \sum_{x \in \mathcal{S}} \widehat{P}_{k,h}(x|s, a) V_k(x, h+1) \\ Q_k(s, a, h) = \min(Y, Q_{\max}, Q_{k-1}(s, a, h)) \end{cases}$$

- $Q_{\max}$ is the the maximum Q-value in the problem (or an upper bound thereof).
- When all rewards belong to $[0, 1]$, then $Q_{\max} = H$.
- Finally, we set π_k to be the greedy policy w.r.t. Q_k .

$$\pi_k(s, h) = \operatorname{argmax}_{a \in \mathcal{A}} Q_k(s, a, h)$$



UCBVI

Assume rewards bounded in $[0, 1]$, so that $Q_{\max} = H$.

- **input:** Failure probability δ
- **initialization:** $\mathcal{D} = \{\}$
- **for** episodes $k = 1, 2, \dots$ **do**

1. *Policy Selection*

- Update counters $N(s, a, h)$ using $\mathcal{D}$
- Compute estimates $\hat{P}$ and $\hat{R}$ using $\mathcal{D}$
- Compute bonus explorations $b(s, a, h)$
- Set $V(s, H + 1) = 0$ for all s
- **for** timesteps $h = H, H - 1, \dots, 1$ **do**

1. Set for all (s, a) :

$$\left\{ \begin{array}{l} Y = \hat{R}_h(s, a) + b(s, a, h) + \sum_{x \in \mathcal{S}} \hat{P}_h(x|s, a)V(x, h + 1) \\ Q(s, a, h) = \min(Y, H, Q(s, a, h)) \end{array} \right.$$

2. Set for s : $V(s, h) = \max_{a \in \mathcal{A}} Q(s, a, h)$

2. *Policy Execution*

- **for** timesteps $h = 1, \dots, H$ **do**
 - Take action $a_h = \arg \max_{a \in \mathcal{A}} Q(s_h, a, h)$
 - Receive reward $r_h \sim R_h(s_h, a_h)$ and next-state $s_{h+1} \sim P_h(\cdot | s_h, a_h)$
 - $\mathcal{D} \leftarrow \mathcal{D} \cup \{(h, s_h, a_h, r_h)\}$



Randomized Least-Square Value Iteration

RLSVI = Randomized Least-Square Value Iteration

- The original idea of RLSVI was introduced in (Osband et al., 2016).
- In the frequentist setting is studied and analyzed in the frequentist setting in (Russo, 2019).
- A model-based RL algorithm that performs exploration by *randomizing value function estimates*, rather than adding explicit exploration bonuses (e.g., UCBVI).
- It has strong connections to implementing posterior sampling in the domain of MDPs (and therefore is related to the Thompson Sampling algorithm for bandits).
- Deep version: Bootstrapped DQN (Osband et al., 2016)



RLSVI: Idea

Consider the empirical MDP: $\widehat{M}_k = (\mathcal{S}, \mathcal{A}, \widehat{P}_k, \widehat{R}_k)$

Idea: To encourage exploration, we perturb $\widehat{M}_k$ by adding a **noise vector** w_k to the rewards

$$\widetilde{M}_k = (\mathcal{S}, \mathcal{A}, \widehat{P}_k, \widehat{R}_k + w_k)$$

Specifically, $w_k \in \mathbb{R}^{SAH}$ with

$$w_k(s, a, h) \sim \mathcal{N}\left(0, \frac{\beta_k}{N_k(s, a, h) + 1}\right)$$

- $\mathcal{N}(0, \sigma^2)$ is zero-mean Gaussian distribution with variance σ^2 .
- β_k can be tuned, but for theoretical guarantees, $\beta_k = \frac{1}{2}SH \log(2SHK)$.



RLSVI

- **input:** $(\beta_k)_k$
- **initialization:** $\mathcal{D} = \{\}$
- **for** episodes $k = 1, 2, \dots$ **do**
 - 1. *Policy Selection*
 - Update counters $N(s, a, h)$ using $\mathcal{D}$
 - Compute estimates $\hat{P}$ and $\hat{R}$ using $\mathcal{D}$
 - Draw $w \sim \mathcal{N}\left(0, \frac{\beta_k}{N(s, a, h) + 1}\right)$
 - $\pi_k = \text{DP}(\widetilde{M})$ where $\widetilde{M} = (\mathcal{S}, \mathcal{A}, \hat{P}, \hat{R} + w)$
 - 2. *Policy Execution*
 - **for** timesteps $h = 1, \dots, H$ **do**
 - Take action $a_h = \pi_k(s_h, h)$
 - Receive reward $r_h \sim R_h(s_h, a_h)$ and next-state $s_{h+1} \sim P_h(\cdot | s_h, a_h)$
 - $\mathcal{D} \leftarrow \mathcal{D} \cup \{(h, s_h, a_h, r_h)\}$

For theoretical guarantees, set $\beta_k = \frac{1}{2}SH \log(2SHK)$.



RLSVI and UCBVI: Regret Guarantees



UCBVI: Regret

Theorem (Regret of UCBVI)

Let $\delta \in (0, 1)$. The regret under UCBVI satisfies

$$\mathfrak{R}(K) \leq 20H^2L\sqrt{SAK} + 250H^3S^2AL^2.$$

with probability at least $1 - \delta$, with $L = \log(5H^3SAK/\delta)$.

$\implies$ UCBVI is **no-regret** non-asymptotically.

Note this is a worst-case bound:

$$\sup_{M' \in \mathbb{M}} \mathfrak{R}_{M'}(K) \leq 20H^2L\sqrt{SAK} + 250H^3S^2AL^2, \quad \text{w.p.} \geq 1 - \delta$$

where $\mathbb{M}$ is the set of all episodic MDP with S states, A actions, episode length H .



RLSVI: Regret

Theorem (Regret of RLSVI)

$\beta_k = \frac{1}{2}SH \log(2SHK)$. The regret under RLSVI satisfies

$$\mathbb{E}[\mathfrak{R}(K)] \leq O\left(H^3 S^{3/2} \sqrt{AK}\right)$$

$\implies$ RLSVI is **no-regret** non-asymptotically.



Worst-Case Lower Bound

Theorem (Worst-Case Regret Lower Bound)

For *any learning algorithm* $\mathbb{A}$, there exists *an MDP* M with S states, A actions, and episode length H such that the regret under $\mathbb{A}$ satisfies

$$\mathbb{E}[\mathfrak{R}(T)] \geq \Omega(\sqrt{H^2 SAT}) = \Omega(\sqrt{H^3 SAK})$$

Compare it to the minimax lower bound of $\Omega(\sqrt{KT})$ for stochastic K -armed bandits.



References

- Tze Leung Lai and Herbert Robbins, "Asymptotically efficient adaptive allocation rules," *Advances in Applied Mathematics*, 1985.
- Mohammad G. Azar, Ian Osband, and Remi Munos, "Minimax regret bounds for reinforcement learning," in *Proc. International Conference on Machine Learning*, 2017.
- Ian Ostand, Charles Blundell, Alexander Pritzel, and Benjamin Van Roy, "Deep exploration via bootstrapped DQN," *Advances in Neural Information Processing Systems*, 2016.
- Dan Russo, "Worst-case regret bounds for exploration via randomized value functions," *Advances in Neural Information Processing Systems*, 2019.

